6_24_marzo

(Continuo TCP vedi 5_19_marzo)

Timer RTT

è importante che il timer venga stimato nel modo corretto, perché:

- Se troppo piccolo: i pacchetti non arrivano in tempo
- Se troppo lungo: si ha una reazione lenta alla perdita di pacchetti

Come stimiamo RTT?

- *Sample RTT*: il tempo trascorso tra l'invio del pacchetto e la ricezione dell'ACK, basandoci su questo andiamo a calcolare *Estimated RTT* come di seguito:
- *Estimated RTT*:

$$\text{Estimated_RTT}_n = (1-\alpha) * \text{Estimated_RTT}_{n-1} + \alpha * \text{Sample_RTT}_n$$

  il valore tipico di α è 0.125

Ovvero il tempo RTT stimato dopo l'arrivo del pacchetto numero n viene calcolato usando il tempo stimato precedente e RTT dell'ultimo pacchetto arrivato

La natura variabile del RTT ci porta a voler tener conto anche della deviazione standard dei valori (come già visto nella scorsa lezione)

$$\text{DevRTT} = (1-\beta) \cdot \text{DevRTT} + \beta \cdot |\text{SampleRTT} - \text{EstimatedRTT}|$$

Usando questo valore andiamo a calcolare il tempo massimo di ritrasmissione **RTO(Retransmission TimeOut)** come di seguito:

$$RTO = \text{EstimatedRTT} + 4 * DevRTT$$

Nonostante RDT stabilisca che ad un pacchetto venga associato un timer, TCP riduce la complessità di gestione gestendo esclusivamente un timer alla volta. Dopo la ricezione dell'ACK dell'n-esimo pacchetto, il timer per il segmento n + 1 non ancora acknowledged viene avviato.

Per ridurre il tempo di attesa nasce la tecnica **Fast Retransmit**: se il reciver manda tre ACK consecutivi per lo stesso pacchetto mancante (ovvero l'ACK dell'ultimo pacchetto consegnato con successo), viene attivata istantaneamente la ritrasmissione del pacchetto corrente.

Controllo del flusso

Cosa succede se il ricevitore non è abbastanza veloce a riceve i pacchetti? Semplicemente viene usato un campo del pacchetto ACK chiamato *receive window* dove viene specificato dal mittente lo spazio massimo disponibile.

Per gestire in modo ottimale questa finestra usiamo un algoritmo detto *Nagle Algorithm* che sarebbe questo:

```
if available_data > 0 then
    if window_size ≥ MSS & available_data ≥ MSS then
        send_a_MSS_segment
    else
        if waiting_for_an_ack == true then
            enqueue_data /* until an acknowledge is received */
        else
            send_data
        end if
    end if
end if
```

Questo algoritmo si comporta in modo diverso in base alla rete in cui si trova:

- In reti con un basso RTT, questo algoritmo invia piccoli pacchetti con alta frequenza
- In reti con un alto RTT, i dati vengono bufferizzati e inviati in grandi pacchetti Alcune volte per avere una forte reattività il sistema operativo disabilita questo algoritmo

Apertura della connessione

La connessione TCP funziona con il 3-way-handshake, ma prima di vederlo capiamo anche perché il 2 non viene utilizzato.

2-way-handshake

Inizialmente si era pensato di stabilire una connessione usando questo metodo che si ispira un po' al modo di fare umano, ma questa metodologia nel mondo dei computer può causare problemi di Half-connection

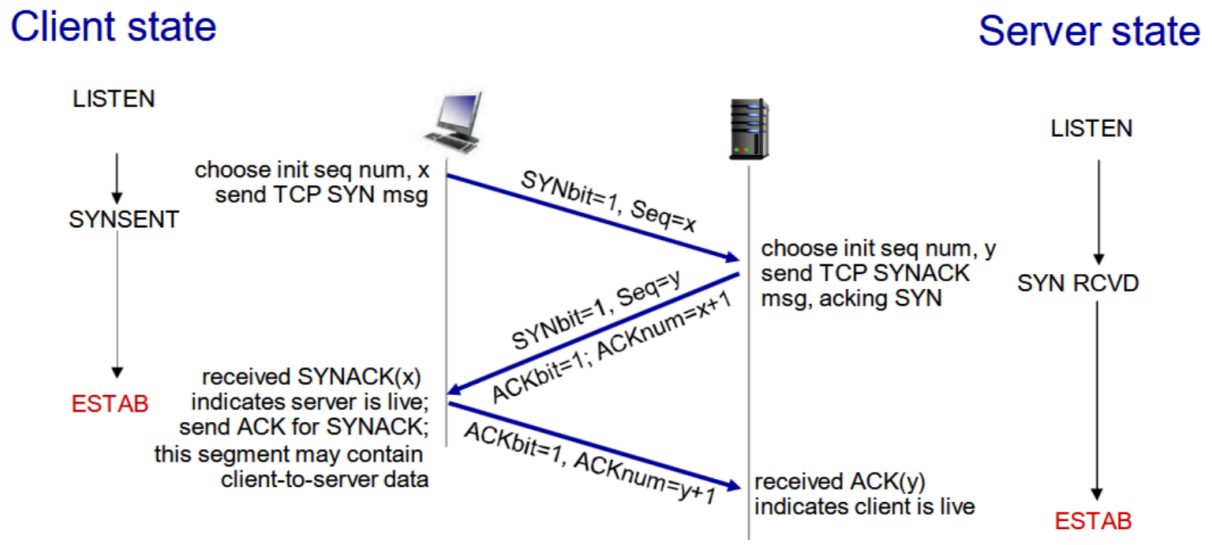
1. A manda una richiesta di connessione a B
2. B manda un ACK e apre la connessione
3. A riceve l'ACK e partecipa alla connessione

e se l'ACK si perde?

1. A manda una richiesta di connessione a B
2. B manda un ACK e apre la connessione
3. L'ACK si perde o A crasha, B rimane così con una connessione aperta ma inutilizzabile

3-way-handshake

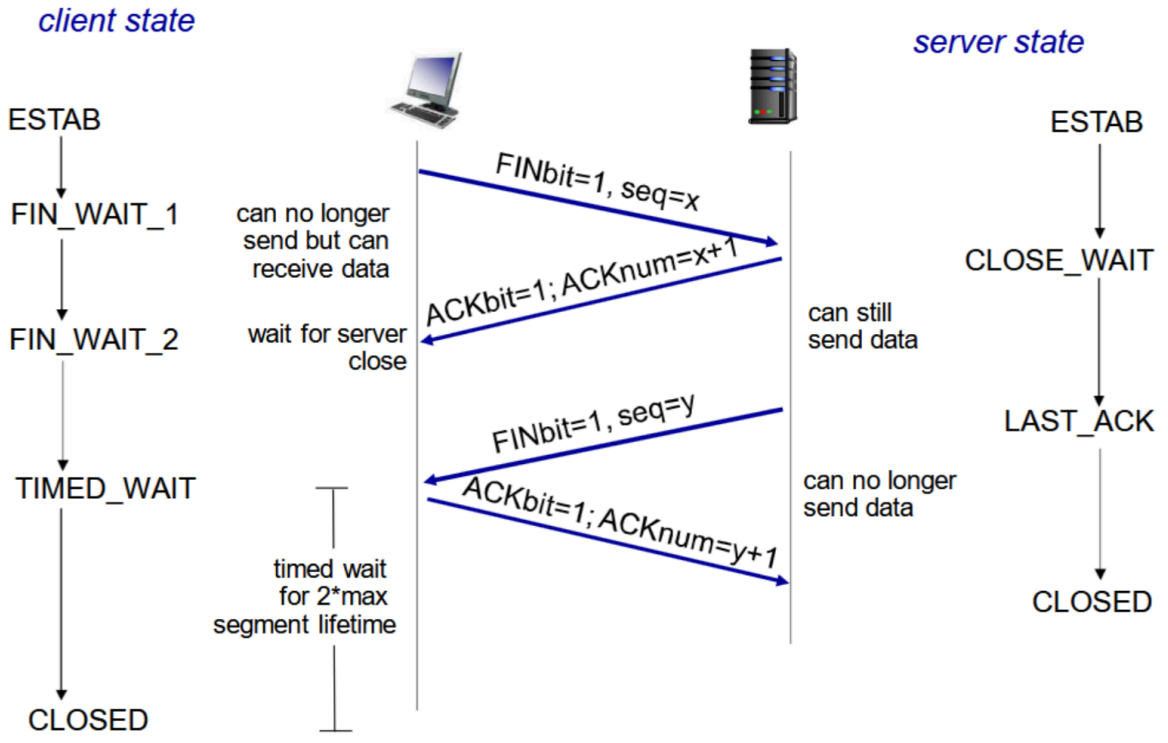
Questo è il metodo scelto da TCP per funzionare correttamente



TCP è un protocollo di connessione full-duplex.
Quindi il client manda una richiesta di connessione (vuole aprire il canale) al server, il server manda un ACK in cui accetta la connessione e contemporaneamente chieda al client di aprire il suo canale verso di lui (full-duplex) e allora il client manda un altro ACK per indicare che l'ACK precedente è stato ricevuto e la connessione funziona.
Eventuali problemi di ACK non ricevuti si risolvono rimandando gli ACK precedenti (in caso non fossero arrivati) per un numero di tentativi limitato, se non si ha successo si interrompe la connessione

Chiusura della connessione con 4-way-handshake (2x2 handshake)

Per chiudere la connessione TCP usa un doppio 2-way-handshake in questo modo:



Il client quindi manda un messaggio con FINbit = 1 questo significa che non ha altri dati da inviare, il server manda un ACK (per dire che ha capito), dopo se il server ha ancora dati da trasmettere li trasmette (perché è il client che non ne deve inviare più), appena finisce il serve manda anche lui un FINbit = 1, il client manda un ACK per dire che ha ricevuto tutto e si chiude la connessione

Controllo di congestione

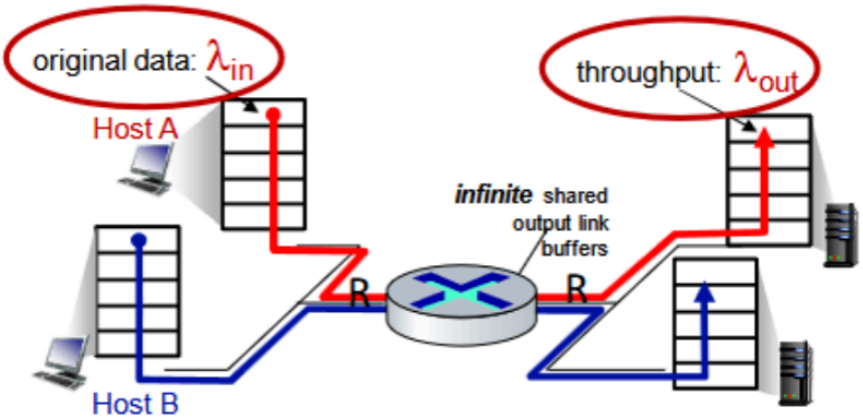
La gestione delle trasmissioni è fondamentale per evitare non solo di sovraccaricare i receiver (eventualità gestita tramite il controllo del flusso), ma anche per evitare il sovraccarico della rete, con conseguente congestione. Il protocollo TCP gode di meccanismi per gestire la congestione, per ridurre la velocità di trasmissione e minimizzare i danni della congestione.

Abbiamo 3 scenari:

Scenario 1 - due mittenti e receiver, router con buffer illimitato (irreale)

Nell'analisi della congestione di rete, è importante conoscere dei valori, quali
- λin, frequenza media di invio (dai mittenti).
- λout, la frequenza di arrivo di byte al ricevente.
- R, la capacità del collegamento condiviso.

Quando $\lambda in \geq \lambda out$, inizia a crearsi congestione.
Consideriamo due host mittenti A e B che inviano a due host destinatari attraverso un router intermedio, e supponiamo che il suo buffer sia illimitato (cosa che nella realtà è impossibile).



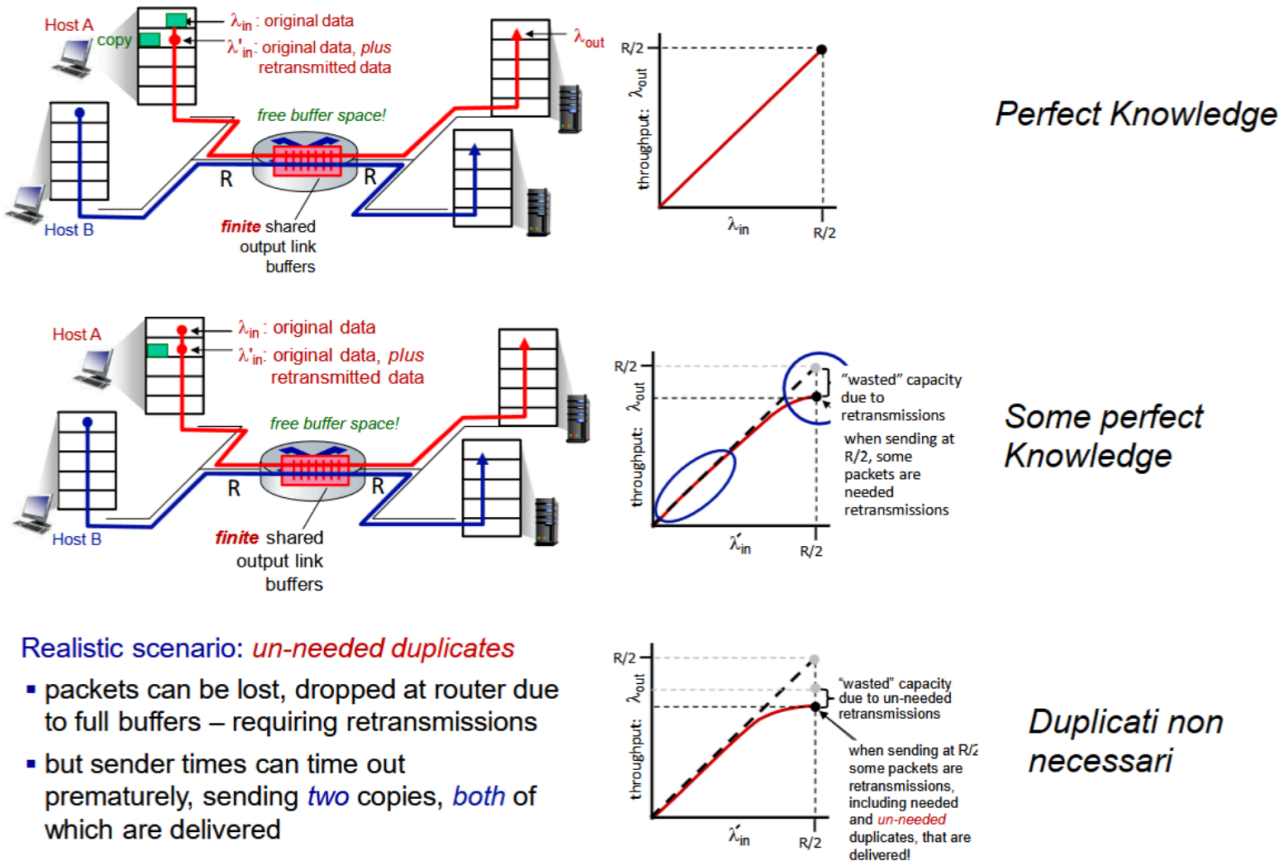
• Se λin ≤ $\frac{R}{2}$, il router riesce ad inoltrare i pacchetti senza ritardi significativi, e λout = λin. Dividiamo R in due perché abbiamo due host che mandano.

• Se λin aumenta fino a $\frac{R}{2}$, il λout massimo si stabilizza su $\frac{R}{2}$.

• Se λin > $\frac{R}{2}$, il throughput non aumenta, ma il numero di pacchetti accodati nel buffer del router cresce senza limite, aumentando il ritardo medio di trasmissione.

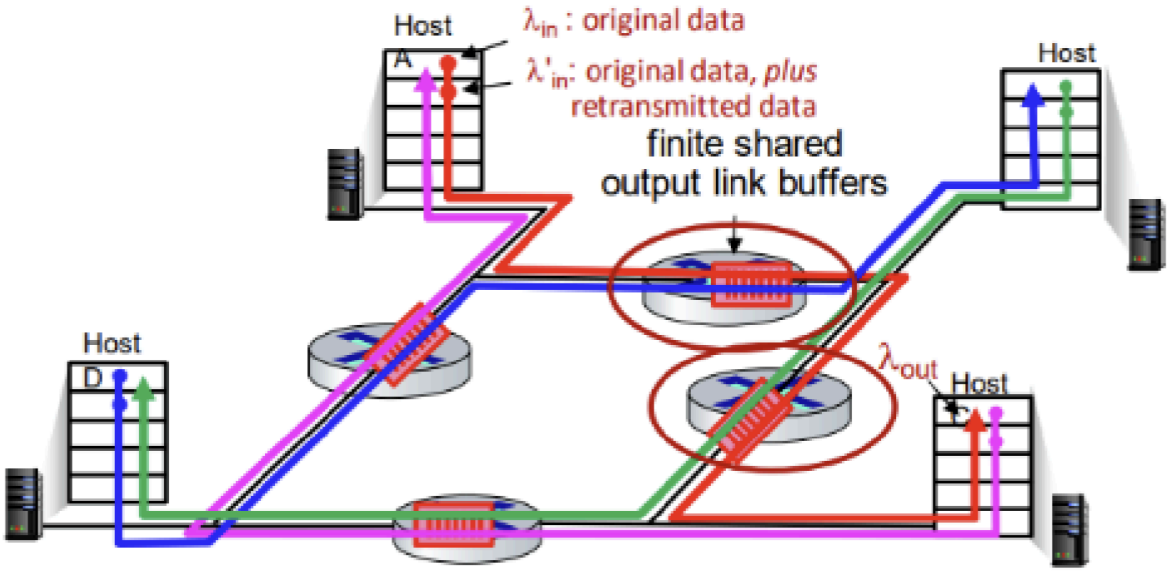
Scenario 2 - due mittenti e receiver, un router con buffer limitato (reale)
Quando il buffer di un router è pieno, alcuni pacchetti vengono scartati. Di fronte a circostanze simili, TCP prevede ritrasmissione, con conseguente aumento della latenza complessiva e riduzione della frequenza media di invio λin, in quanto vengono rimandati gli stessi pacchetti. λ'in tiene conto sia dei dati originali, che delle ritrasmissioni. Distinguiamo ora 3 casi di questo scenario.

1. *Nessuna perdita di pacchetti - Perfect Knowledge*. Se i sender fossero in grado di determinare in anticipo se il router ha spazio disponibile nel buffer, invierebbe un pacchetto solo quando può essere immediatamente elaborato, ottenendo λin = λ'in, λout = λin ≤ $\frac{R}{2}$ e zero latenza (perlomeno, causata dal router).
2. *Ritrasmissione solo per pacchetti persi - Some perfect Knowledge*. I sender non conoscono lo stato del buffer, e ritrasmettono un pacchetto solo quando hanno la certezza che sia stato perso. Supponendo che λ'in =$\frac{R}{2}$, λout < $\frac{R}{2}$ a causa delle ritrasmissioni.
3. *Caso reale - duplicati non necessari*. I sender non conoscono lo stato del buffer, e potrebbero avere un timer di ritrasmissione troppo breve. Questo implica un utilizzo innecessario del buffer del router, della banda e delle risorse di elaborazione perché avendo un tempo troppo breve potrebbe mandare dei duplicati. Questo potrebbe addirittura portare ad un throughput di circa λout = $\frac{R}{4}$.



Terzo scenario - quattro mittenti, più router limitati e percorsi

In questo scenario, quattro mittenti inviano pacchetti su percorsi composti da più collegamenti, condividendo router intermedi. Ogni host utilizza un meccanismo di ritrasmissione basato su time-out, e la rete ha buffer di dimensione finita.

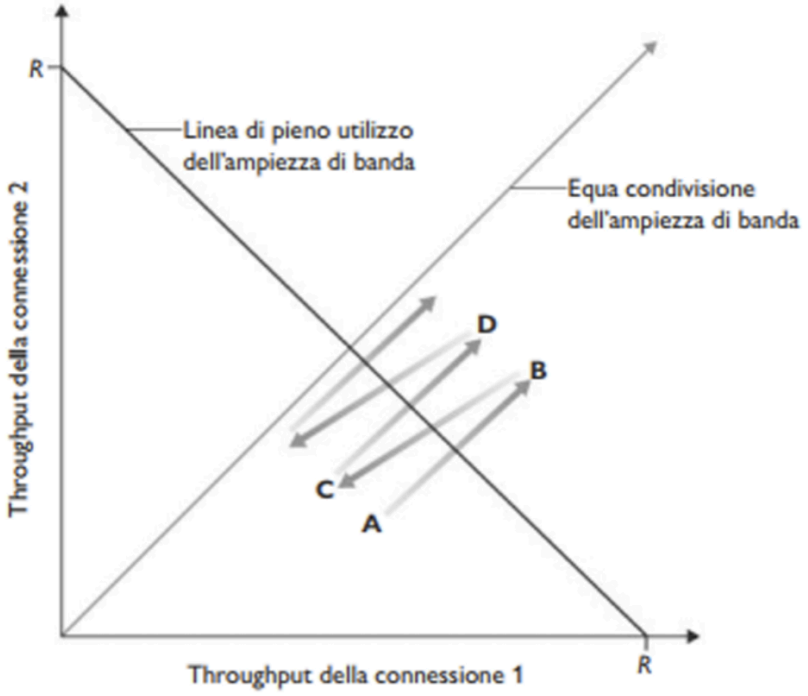


Abbiamo quattro sender che inviano pacchetti in quattro percorsi con più router, e quindi più possibili percorsi. Ogni router trasmette a capacità R. A valori di λin piccoli, λin = λ'in, non si hanno overflow dei buffer e nemmeno ritrasmissioni. Al crescere del valore di λin, inizia il rischio di ritrasmissioni, per cui λ'in > λin.

TCP Fairness

TCP è un protocollo fair? Cosa intendiamo per fairness?
Si desidera che il controllo della congestione in TCP sia tale che, date K connessioni attraverso la stessa rete con capacità trasmissiva R, la *cwnd* di ogni connessione sia $\frac{R}{K}$ in modo che ogni host abbia la stessa quantità di banda degli altri

Dimostrazione
Date due connessioni con pari MSS (Maximum Segment Size) e RTT (tempo di propagazione andata e ritorno) (e nessun'altra connessione attraverso il collegamento) che operano in *congestion avoidance*. Con un throughput massimo pari a R, e tenendo a mente che al crescere di uno (connessione 1 ha più spazio), diminuisce l'altro (connessione 2 ha meno spazio), il throughput in una situazione di equilibrio è pari a $\frac{R}{2}$



Nel punto in cui si incontrano le rette perpendicolari si ha una perfetta divisione di banda tra le 2 connessioni.

A parità di RTT e MSS, gli host in congestion avoidance crescono in maniera lineare: 1 MSS ad ogni RTT . Detto ciò, quando la somma del throughput di entrambi gli host supera R, questi si dimezzano. Con $x = throughput$ del primo host e $y = throughput$ del secondo host, e il comporta- mento di incremento lineare e divisione della finestra di congestione, il throughput delle connessioni convergerà sempre alla bisettrice x = y. In circostanze più vicine a quelle reali, le connessioni con RTT più basso ottengono un throughput maggiore a causa della velocità con cui possono ottenere la connessione. Ricordiamo che l'incremento, nella fase di congestion avoidance, avviene una volta per RTT.